

Lekce 17: Problémy a řešení v reálném světě s Cypress – příklady z praxe

Cíle

- Identifikovat běžné výzvy při používání Cypressu pro end-to-end testování v reálných aplikacích.
- Poskytnout praktická řešení a strategie pro práci s dynamickým obsahem, asynchronním chováním, nestabilními testy a složitými interakcemi.
- Sdílet osvědčené postupy pro psaní udržitelných, škálovatelných a spolehlivých testů.
- Představit případové studie z reálného světa a příklady, zapojit se do praktických aktivit zaměřených na řešení a refaktoring problémových testů.

Obsah lekce

A. Běžné výzvy v Cypressu

1. Práce s dynamickým obsahem a asynchronním chováním

- **Problém:**

Mnoho webových aplikací aktualizuje UI dynamicky, což může vést k tomu, že prvky nejsou okamžitě dostupné v době, kdy je test očekává.

- **Výzvy:**

- Čekání na zobrazení prvků.
- Zajištění, že asynchronní API volání a animace jsou dokončeny před ověřením.
- Práce s nekonzistentními stavy prvků.

2. Práce s nestabilními testy a zajištění spolehlivosti testů

- **Problém:**

Nestabilní testy jsou ty, které někdy projdou a jindy selžou bez změn v kódu, často kvůli načasování, zpoždění sítě nebo nestabilním selektorům.

- **Výzvy:**

- Nekonzistentní načasování vykreslení prvků.
- Dočasné síťové podmínky.
- Rozdíly mezi lokálním a CI prostředím.

3. Správa složitých uživatelských interakcí a stavových aplikací

- **Problém:**

Aplikace se složitými workflow, více stavy nebo vnořenými komponentami mohou být těžko

testovatelné spolehlivě.

- **Výzvy:**

- Simulace vícestupňových uživatelských interakcí (např. drag-and-drop, vnořené modální okno).
- Udržení aplikačního stavu napříč testy.
- Řešení interakcí závislých na datech z více zdrojů nebo API odpovědí.

B. Praktická řešení a strategie

1. Efektivní strategie čekání

- **cy.wait():**

Použijte `cy.wait()` , pokud je nezbytné pozastavit provádění na pevně stanovený čas.

Příklad:

```
cy.wait(1000); // Čekání 1 sekundu
```

- **Chytrá ověření:**

Využijte automatický mechanismus opakovaných pokusů Cypressu pomocí ověřování přítomnosti či stavu prvků.

Příklad:

```
cy.get('[data-testid="dynamic-element"]').should('be.visible');
```

- **Vlastní čekání pomocí cy.waitUntil:**

Využijte plugin `cypress-wait-until` k čekání na splnění podmínky.

Příklad:

```
cy.waitUntil(() => cy.get('[data-testid="status"]').should('contain', 'Ready'));
```

- **Proč se vyhýbat explicitnímu cy.wait()**

1. Nestabilita a nepředvídatelnost testu:

- **Pevné čekání je svévolné:**

Explicitní čekání po pevnou dobu (např. `cy.wait(2000)`) předpokládá, že aplikace je vždy připravena po uplynutí této doby. Ve skutečnosti se mohou měnit latence sítě, časy odpovědi serveru nebo vykreslení na klientovi. Testy pak mohou někdy projít, jindy selhat – vzniká nestabilita (flakiness).

- **Promarněný čas:**

Pevná čekání zbytečně prodlužují čas testování i v případě, že aplikace je připravena dříve.

Pokud se prvek načte za 500 ms, ale čekáte 2000 ms, ztrácíte čas.

2. Lepší alternativy:

- **Timeouty v ověřeních:**

Vestavěná ověření typu `.should()` mají implicitní mechanismus opakování. Pokud ověřujete, že prvek je viditelný pomocí `.should('be.visible')`, Cypress opakuje ověření, dokud prvek nezobrazí nebo nevyprší výchozí timeout.

Příklad:

```
// Cypress opakuje ověření, dokud se prvek nezobrazí nebo nevyprší výchozí timeout (4 s)
cy.get('[data-testid="login-button"]').should('be.visible');
```

- **Plugin `cy.waitUntil()`:**

Plugin `cypress-wait-until` nabízí dynamičtější mechanismus čekání – opakovaně kontroluje podmínku, dokud není splněna. Tím vyhnete svévolným čekacím dobám a testy jsou spolehlivější.

Příklad:

```
// Čeká, dokud prvek status nebude obsahovat text "Ready"
cy.waitUntil(() => cy.get('[data-testid="status"]').should('contain', 'Ready'), {
  timeout: 10000, // Čekání max. 10 sekund
  interval: 500,  // Kontrola každých 500 ms
});
```

2. Snížení nestability pomocí chytrých selektorů a opakování

- **Chytré selektory:**

Používejte stabilní selektory jako `data-testid` nebo `data-cy` místo křehkých CSS tříd nebo pozic.

- **Opakování testů:**

Nakonfigurujte opakování testů v Cypressu, aby se nestabilní testy automaticky zopakovaly.

Příklad (globálně v `cypress.config.js`):

```
retries: {
  runMode: 2,
  openMode: 0,
}
```

- **Úprava timeoutů:**

Zvyšte výchozí timeouty při práci s pomalu načítanými prvky nebo externími API.

3. Správa aplikačního stavu

4. `cy.session`: Správa session napříč testy

- **Účel:**

cy.session ukládá a obnovuje stav (např. cookies, localStorage), díky čemuž není nutné opakovaně provádět stejné kroky (jako přihlášení) napříč testy.

- **Příklad použití:**

```
// Příklad: Použití cy.session k uložení přihlášení
cy.session('loginSession', () => {
  cy.visit('/login');
  cy.get('[data-testid="login-username-input"]').type('demoUser');
  cy.get('[data-testid="login-password-input"]').type('demoPass');
  cy.get('[data-testid="login-submit-button"]').click();
  cy.get('[data-testid="login-success-message"]').should('be.visible');
});

// Později v testech se session automaticky obnoví
cy.visit('/dashboard');
```

- **Výhody:**

Zrychlení testů tím, že se předejde opakovanému přihlašování; zajištění konzistentního stavu testů.

Význam cy.session() z výkonnostního hlediska

Co je cy.session() ?

cy.session() ukládá a obnovuje session stav (např. cookies, localStorage, sessionStorage) mezi testy. To je praktické zejména při multidoménovém testování a optimalizaci výkonu.

Význam při multidoménovém testování:

- **Konzistentní session napříč doménami:**

Když testy vyžadují přihlášení nebo session stav přes více domén, cy.session() umožňuje zachovat autentizovaný stav. Například, pokud aplikace autentizuje na jedné doméně a uživatel je přesměrován na druhou, ukládání session může testy zjednodušit.

- **Snížení overheadu při přípravě:**

Místo přihlašování před každým testem lze session připravit jednou a opakovaně ji využít. To je velmi užitečné v komplexních procesech napříč doménami.

Z pohledu výkonu:

- **Rychlejší provádění testů:**

Díky ukládání dat session jsou testy rychlejší, protože není nutné opakovat přihlašování. Zkrátí se doba běhu celé testovací sady.

- **Stabilita:**

Konzistentní session napříč testy znamenají méně nestabilních testů souvisejících s

nekonzistentními přihlášeními nebo výpadky sítě.

Příklad použití `cy.session()` v multidoménovém scénáři:

```
describe('Dashboard Tests with Session Management', () => {
  // Vytvoření session s uložením přihlašovacího procesu včetně průchodu více doménami
  beforeEach(() => {
    cy.session('loginSession', () => {
      cy.visit('/login');
      cy.get('[data-testid="login-with-oauth"]').click();

      cy.origin('https://auth.example.com', () => {
        cy.get('[data-testid="auth-username"]').type('externalUser');
        cy.get('[data-testid="auth-password"]').type('externalPass');
        cy.get('[data-testid="auth-submit"]').click();
      });

      cy.url().should('include', '/dashboard');
      cy.get('[data-testid="welcome-message"]').should('contain', 'Welcome');
    });
  });

  it('should display dashboard elements using cached session', () => {
    cy.session('loginSession'); // Obnovit session
    cy.visit('/dashboard');
    cy.get('[data-testid="dashboard-title"]').should('be.visible');
  });
});
```

- **`cy.session()`:**

Použijte `cy.session()` k uložení přihlašovacích session nebo opakovaných setup kroků. Tím snížíte zbytečné operace a zajistíte konzistenci.

```
cy.session('userSession', () => {
  cy.visit('/login');
  cy.login('demoUser', 'demoPass');
  cy.get('[data-testid="login-success-message"]').should('be.visible');
});
```

- **Vlastní příkazy:**

Abstrahujte složité interakce do vlastních příkazů pro centralizovanou správu stavu.

```
Cypress.Commands.add('login', (username, password) => {  
  cy.get('[data-testid="login-username-input"]').clear().type(username);  
  cy.get('[data-testid="login-password-input"]').clear().type(password);  
  cy.get('[data-testid="login-submit-button"]').click();  
});
```

5. Práce se složitými uživatelskými interakcemi

- **cy.origin():**

Při multidoménových nebo cross-origin interakcích použijte `cy.origin()` pro přepnutí kontextu.

Příklad:

```
cy.origin('https://external.example.com', () => {  
  cy.get('[data-testid="external-element"]').should('be.visible');  
});
```

C. Osvědčené postupy

1. Psaní udržitelných a škálovatelných testů

- Používejte modulární testovací struktury (např. vnořené bloky `describe`).
- Držte testy co nejvíce nezávislé a bezstavové.
- Seskupujte podobné testy do kontextů a používejte vlastní příkazy pro běžné akce.

2. Organizace testovacího kódu pro čitelnost a znovupoužitelnost

- Oddělujte logiku testů od pomocných funkcí.
- Používejte smysluplná jména testů, kontextů i vlastních příkazů.
- Mějte přehlednou strukturu adresářů pro testovací soubory, vlastní příkazy a fixturny.

3. Využití funkcí Cypressu

- Využívejte vestavěné mechanismy opakování a čekání při práci s dynamickým obsahem.
- Používejte pluginy Cypressu (např. `cypress-wait-until`, `cypress-drag-drop`, `cypress-axe` apod.) podle potřeby.
- Konfigurujte globální nastavení (např. `baseUrl`, `timeouty`, `retries`) pro zvýšení stability testů.

D. Případové studie a příklady z praxe

1. Práce s dynamickým obsahem:

- **Scénář:**

Stránka načítá seznam položek až po API volání.

- **Řešení:**

Použijte chytré ověření pro čekání na vykreslení seznamu a v případě nutnosti opakujte pokus.

- **Příklad:**

```
cy.get('[data-testid="item-list"]').should('be.visible');
```

2. Nestabilní test kvůli pomalé síti:

- **Scénář:**

API odpověď přijde někdy pomaleji, než očekáváte.

- **Řešení:**

Nastavte delší timeout pro konkrétní příkaz nebo použijte vlastní čekací funkci.

- **Příklad:**

```
cy.get('[data-testid="status"]').should('contain', 'Loaded', { timeout: 10000 });
```

3. Složitá interakce (multidoménové přihlášení):

- **Scénář:**

Uživatel se přihlašuje na jedné doméně a je přesměrován na jinou.

- **Řešení:**

Použijte `cy.origin()` pro přepnutí doménového kontextu a `cy.session()` pro uchování autentizačního stavu.

- **Příklad:**

```
cy.session('loginSession', () => {  
  cy.visit('/login');  
  cy.login('demoUser', 'demoPass');  
  cy.get('[data-testid="login-success-message"]').should('be.visible');  
});
```

```
cy.origin('https://external.example.com', () => {  
  cy.visit('/external-dashboard');  
  cy.get('[data-testid="external-dashboard"]').should('be.visible');  
});
```

E. Praktické aktivity

1. Analýza a řešení problémů studentských projektů:

- Otevřená diskuze, během níž lze demonstrovat a řešit reálné problémy v osobních projektech studentů.

F. Další zdroje

- **Dokumentace osvědčených postupů Cypress:**

[Cypress Best Practices](#)

- **Případové studie:**

Hledejte blogy a repozitáře na GitHubu, které popisují, jak týmy řešily nestabilní testy a problémy s dynamickým obsahem.

Problémy v reálném světě při testování Cypressem

1. Nestabilní testy kvůli asynchronnímu chování:

- **Problém:**

Testy někdy selžou, protože prvky se zobrazí či změní stav v nepředvídatelném čase kvůli latenci sítí nebo dynamickým aktualizacím.

- **Řešení:**

Používejte implicitní čekání s ověřeními (např. `.should('be.visible')`) nebo `cy.waitUntil()` pro dynamičtější čekání místo pevných prodlev.

2. Závodní podmínky:

- **Problém:**

Pokud probíhá více asynchronních akcí (např. současná API volání), test může ověřit stav dříve, než jsou všechny akce dokončeny.

- **Řešení:**

Používejte `cy.intercept()` pro čekání na odpovědi API (`cy.wait('@alias')`) a řetězte ověření až po dokončení všech požadovaných akcí.

3. Problémy s výběrem prvků:

- **Problém:**

Dynamické aplikace mohou měnit identifikátory prvků, což může způsobit selhání testů, pokud selektory vycházejí z CSS tříd či pozic.

- **Řešení:**

Používejte stabilní selektory jako `data-testid` nebo `data-cy`, které se pravděpodobněji nezmění.

4. Správa session a stavu:

- **Problém:**

Testy závislé na stavu aplikace (například přihlášený uživatel) mohou selhat, pokud není správně resetován nebo sdílen mezi testy.

- **Řešení:**

Ukládejte a obnovujte session data pomocí `cy.session()`, aby byl zajištěn konzistentní výchozí stav.

5. Práce s více doménami:

- **Problém:**

Při testování aplikací napříč více doménami můžete narazit na problémy týkající se politiky stejného původu (same-origin policy).

- **Řešení:**

Využijte `cy.origin()` pro bezpečné interakce s prvky na externích doménách bez porušení politiky stejného původu.

6. Variabilita prostředí:

- **Problém:**

Rozdíly mezi lokálním, testovacím a produkčním prostředím mohou způsobit nestabilní chování testů.

- **Řešení:**

Používejte proměnné prostředí (nastavené v `.env` souborech a načítané pomocí prefixu `VITE_`) pro správu `baseUrl`, přihlašovacích údajů nebo endpointů API. Tak mohou být testy adaptabilní různým prostředím.

7. Nestabilita sítě:

- **Problém:**

Nepředvídatelné reakce sítě mohou způsobit nestabilitu testů závislých na API.

- **Řešení:**

Používejte `cy.intercept()` pro stubování odpovědí sítí, zajištění konzistentních dat v testech a omezení závislosti na externích systémech.

Možné otázky studentů a odpovědi

1. **Q:** *Proč jsou některé testy nestabilní a jak tomu předejít?*

A: Testy mohou být nestabilní kvůli asynchronnímu obsahu, zpožděním sítě nebo dynamickým prvkům. Omezte nestabilitu používáním chytrých ověření, vhodných čekacích časů, opakování a stabilních selektorů. Pomoci mohou pluginy, jako je `cypress-wait-until`.

2. **Q:** *Jaký je význam `cy.origin()` při multidoménovém testování?*

A: `cy.origin()` umožňuje spouštět příkazy na jiné doméně, než je hlavní aplikace. Tím můžete testovat scénáře, kdy je nutné překročit hranici domény (například přihlášení třetí stranou), bez narušení politiky stejného původu.

3. **Q:** *Jak `cy.session()` zlepšuje výkon testů v multidoménových scénářích?*

A: `cy.session()` ukládá a obnovuje session data (například cookies a localStorage), což omezuje potřebu opakovaného přihlašování nebo nastavení prostředí. Testy jsou pak rychlejší a spolehlivější, hlavně u interakcí napříč doménami.

4. **Q:** *Jaké jsou osvědčené postupy pro organizaci testů s cílem předejít nestabilitě?*

A: Pište modulární testy s vnořenými bloky `describe`, používejte stabilní selektory (`data-testid`), nastavte vhodné timeouty a leverage retries. Udržujte testy oddělené a nezávislé.

5. **Q:** *Jakou roli hrají pluginy při řešení problémů v Cypressu?*

A: Pluginy přidávají další možnosti (například pokročilé čekání, kontroly přístupnosti nebo rozšířené logování), které pomáhají řešit dynamický obsah a nestabilní testy. Snižují množství boilerplate kódu a dělají testy robustnějšími.

6. **Q:** *Jak poznat, kdy použít `cy.wait()` vs. chytřejší strategie jako `cy.waitUntil()`?*

A: `cy.wait()` použijte pro pevné prodlevy v předvídatelných situacích. U dynamického obsahu je lepší použít `cy.waitUntil()` nebo řetězení ověření s vestavěnou retry logikou, protože čekají, dokud podmínka nebude splněna, aniž by zdržovaly test zbytečně dlouho.